

Report: Research-Oriented University Information System

1. Introduction

This project is a console-based Java application that models an information system for a research-oriented university. The system was developed according to the principles of Object-Oriented Programming and includes users, courses, marks, lessons, research papers, research projects, authentication, messaging, requests, logging, and data storage.

The main purpose of the project is to demonstrate correct usage of OOP concepts such as encapsulation, inheritance, polymorphism, abstraction, interfaces, enumerations, collections, custom exceptions, serialization, and design patterns.

The system supports different types of users: Admin, Student, Teacher, Manager, and Employee. Each role has its own responsibilities and menu options. Students can register for courses and view transcripts. Teachers can manage courses and put marks. Managers can manage course registration and requests. Admins can manage users and logs. The research module supports researchers, research papers, sorting papers, and joining research projects.

2. Project Goals

The main goals of the project are:

- To create a university information system using Java and OOP.
- To model the required classes: `User`, `Employee`, `Teacher`, `Manager`, `Student`, `Admin`, `Course`, `Mark`, `Lesson`, `Researcher`, `ResearchPaper`, and `ResearchProject`.
- To implement course registration, mark management, authentication, research activities, and data storage.
- To use custom exceptions for important business rules.
- To use design patterns such as Singleton, Factory, Mediator, and Data Store.
- To provide a console demo for testing the main functionality.

3. System Architecture

The system is divided into several packages:

- `oopproject.users` contains user classes such as `User`, `Student`, `Teacher`, `Manager`, `Admin`, and `Employee`.
- `oopproject.academic` contains academic classes such as `Course`, `Lesson`, `Marks`, `Transcript`, `Schedule`, `Request`, and `Report`.
- `oopproject.research` contains research-related classes and interfaces such as `Researcher`, `ResearchPaper`, `ResearchProject`, `ResearchService`, and `ResearchPaperComparators`.
- `oopproject.core` contains system-level classes such as `University`, `AuthService`, `DataStore`, `Message`, `MessageMediator`, and `ConsoleUIController`.
- `oopproject.enums` contains enumerations such as `UserType`, `TeacherType`, `ManagerType`, `LessonType`, `ResearchSortType`, and others.
- `oopproject.exceptions` contains custom exception classes.

The central class of the system is `University`. It stores the main collections of users, courses, researchers, research projects, requests, and logs.

```
private final List<User> users = new ArrayList<>();
private final List<Course> courses = new ArrayList<>();
private final List<Researcher> researchers = new ArrayList<>();
private final List<ResearchProject> projects = new ArrayList<>();
private final List<Request> requests = new ArrayList<>();
private final List<Log> logs = new ArrayList<>();
```

4. Description of Main Classes

4.1 User

`User` is the base abstract class for all users in the system. It contains common fields such as `id`, `username`, `password`, `email`, `role`, and `active`.

It also provides common behavior such as checking password, receiving news, sending messages, blocking and unblocking users, and viewing created requests.

```
public abstract class User implements Serializable {  
    protected int id;  
    protected String username;  
    protected String password;  
    protected String email;  
    protected UserType role;  
    protected boolean active = true;  
}
```

The class also overrides `equals()`, `hashCode()`, and `toString()` methods, which are important for comparing users and printing user information.

```
@Override  
public boolean equals(Object object) {  
    if (this == object) {  
        return true;  
    }  
    if (!(object instanceof User user)) {  
        return false;  
    }  
    return id == user.id;  
}
```

4.2 Student

`Student` extends `User`. It represents a student in the university system. A student has GPA, year of study, registered credits, failed courses, schedule, enrollments, transcript, and teacher ratings.

```
public class Student extends User {
    public static final int MAX_CREDITS = 21;

    private double gpa;
    private int year;
    private int credits;
    private int failedCourses;
    private final List<Enrollment> enrollments = new ArrayList<>();
    private Transcript transcript = new Transcript(this);
}
```

The most important method in this class is `registerCourse()`. It checks whether the course is open, whether the student is already registered, and whether the credit limit is exceeded.

```
public void registerCourse(Course course)
    throws CreditLimitExceededException, AlreadyRegisteredException {
    if (course == null) {
        throw new IllegalArgumentException("Course cannot be null.");
    }
    if (!course.isOpen()) {
        throw new IllegalStateException("Course is not open for registration.");
    }

    if (credits + course.getCredits() > MAX_CREDITS) {
        throw new CreditLimitExceededException(
            "Student cannot register for more than " + MAX_CREDITS + " credits."
        );
    }

    course.addStudent(this);
    credits += course.getCredits();
}
```

This method demonstrates business logic and custom exception handling.

4.3 Employee

`Employee` extends `User`. It is a parent class for employees of the university. It contains salary, hire date, department, and employee requests.

```
public class Employee extends User {
    protected double salary;
    protected LocalDate hireDate;
    protected String department;
    protected List<Request> requests = new ArrayList<>();
}
```

This class is useful because `Teacher` and `Manager` are also employees, so common employee data can be reused through inheritance.

4.4 Teacher

`Teacher` extends `Employee` and implements `Researcher`. This means a teacher is a university employee and can also be a researcher.

```
public class Teacher extends Employee implements Researcher {
    private TeacherType title;
    private boolean researcher;
    private String researchSchool = "UNSPECIFIED";
    private final List<Course> courses = new ArrayList<>();
    private final List<ResearchPaper> researchPapers = new ArrayList<>();
}
```

Professors are automatically researchers:

```
public boolean putMark(Student student, Course course, Marks marks) {
    if (!course.getInstructors().contains(this)) {
        System.out.println("Teacher is not assigned to this course.");
        return false;
    }

    Enrollment enrollment = course.findEnrollment(student);
    if (enrollment == null) {
        return false;
    }

    enrollment.setMarks(marks);
    University.getInstance().addLog(this, "MARK_PUT " + course.getCode());
    return true;
}
```

4.5 Manager

`Manager` extends `Employee`. It is responsible for course registration management, assigning teachers, reviewing requests, creating news, and generating reports.

```
public class Manager extends Employee {
    private ManagerType type;
    private final List<News> news = new ArrayList<>();
}
```

Managers can approve or reject requests:

```
public boolean approveRequest(Request request) {
    if (!isManagedRequest(request)) {
        return false;
    }
    request.approve();
    University.getInstance().addLog(this, "REQUEST_APPROVED id=" + request.getRequestId());
    return true;
}
```

4.6 Admin

`Admin` extends `User`. It manages users, courses, researchers, and logs. Admin can add users, remove users, add courses, remove courses, block users, and unblock users.

Admin is important for system control and user management.

4.7 Course

`Course` represents a university course. It contains course name, course code, number of credits, registration status, instructors, lessons, materials, and enrollments.

```
public class Course implements Serializable {
    private String name;
    private String code;
    private int credits;
    private boolean open;
    private final List<Teacher> instructors = new ArrayList<>();
    private final List<Lesson> lessons = new ArrayList<>();
    private final List<Enrollment> enrollments = new ArrayList<>();
}
```

A course can have more than one instructor:

```
public void addInstructor(Teacher teacher) {
    if (teacher != null && !instructors.contains(teacher)) {
        instructors.add(teacher);
    }
}
```

Students are added to a course through enrollments:

```
public void addStudent(Student student) {
    if (student != null && findEnrollment(student) == null) {
        enrollments.add(new Enrollment(student, this));
    }
}
```

4.8 Marks and Mark

`Marks` represents student grades. It stores marks for the first attestation, second attestation, and final exam. It can calculate the total mark.

```
public double getTotal() {  
    return getAtt1() + getAtt2() + getFinalExam();  
}
```

The `Mark` class extends `Marks`, which allows compatibility with the required project class name.

4.9 Lesson

`Lesson` represents a lesson in a course. It uses the `LessonType` enum to define whether the lesson is a lecture or practice.

```
public enum LessonType {  
    LECTURE,  
    PRACTICE  
}
```

4.10 Transcript

`Transcript` is connected to a student and calculates registered credits, completed credits, average mark, GPA, and academic standing.

It uses enrollments and marks to generate academic information.

5. Research Module

The research module is one of the most important parts of the project because the system is designed for a research-oriented university.

5.1 Researcher Interface

`Researcher` is implemented as an interface. This design is flexible because a researcher is not always a separate user class. A teacher, student, or employee can be a researcher if they implement this interface.

```
public interface Researcher extends Serializable {  
    boolean addResearchPaper(ResearchPaper paper);  
    boolean removeResearchPaper(ResearchPaper paper);  
    List<ResearchPaper> getResearchPapers();  
    String getResearcherName();  
}
```

The interface also contains default methods for calculating total citations and h-index.

```
default int getTotalCitations() {  
    return getResearchPapers().stream()  
        .mapToInt(ResearchPaper::getCitations)  
        .sum();  
}
```

5.2 ResearchPaper

`ResearchPaper` stores important information about a scientific paper: title, journal, publisher, pages, citations, publication date, DOI, keywords, and authors.

```
java  
public class ResearchPaper implements Comparable<ResearchPaper>, Serializable {  
    private String title;  
    private String journal;  
    private String publisher;  
    private int pages;  
    private int citations;  
    private LocalDate publishDate;  
    private String doi;  
    private String keywords;  
    private final List<Researcher> authors = new ArrayList<>();  
}
```

It implements `Comparable`, so papers can be compared by citations.

```
java  
@Override  
public int compareTo(ResearchPaper other) {  
    return Integer.compare(other.citations, citations);  
}  
...
```

It also overrides `equals()` and `hashCode()`. If DOI exists, papers are compared by DOI. Otherwise, they are compared by title and publication date.

5.3 ResearchPaperComparators

The system supports sorting research papers by date, citations, and article length.

```
public static final Comparator<ResearchPaper> BY_DATE_PUBLISHED_DESC =  
    Comparator.comparing(ResearchPaper::getPublishDate,  
        Comparator.nullsLast(Comparator.reverseOrder()));  
  
public static final Comparator<ResearchPaper> BY_CITATIONS_DESC =  
    Comparator.comparingInt(ResearchPaper::getCitations).reversed();  
  
public static final Comparator<ResearchPaper> BY_ARTICLE_LENGTH_DESC =  
    Comparator.comparingInt(ResearchPaper::getPages).reversed();
```

This satisfies the requirement to print papers sorted by different criteria.

5.4 ResearchProject

`ResearchProject` contains a topic, project participants, and published papers.

```
public class ResearchProject implements Serializable {  
    private final String topic;  
    private final Set<Researcher> participants = new HashSet<>();  
    private final Set<ResearchPaper> papers = new HashSet<>();  
}
```

Only researchers can join a research project. If a non-researcher tries to join, a custom exception is thrown.

```
public boolean addParticipant(Object candidate) throws NonResearcherException {  
    if (!(candidate instanceof Researcher researcher)) {  
        throw new NonResearcherException("Only researchers can join a research project");  
    }  
    return addParticipant(researcher);  
}
```

5.5 ResearchService

`ResearchService` contains utility methods for research operations. It can get all papers sorted, find papers by year, find papers by journal, get top cited papers, and find the top cited researcher.

```
public static Optional<Researcher> findTopCitedResearcher(  
    Collection<? extends Researcher> researchers) {  
    return researchers.stream()  
        .filter(researcher -> researcher != null && researcher.isResearcher())  
        .max(Comparator.comparingInt(Researcher::getTotalCitations));  
}
```

6. OOP Principles Used

6.1 Encapsulation

Encapsulation is used by making fields private or protected and accessing them through methods. For example, in `Student`, fields such as `gpa`, `year`, and `credits` are private.


```

private double gpa;
private int year;
private int credits;

public int getYear() {
    return year;
}

public double getGpa() {
    return gpa;
}

```

This protects the internal state of objects.

6.2 Inheritance

Inheritance is used to create a hierarchy of users.

public class Student extends User

This avoids code duplication because common user fields and methods are stored in the parent class `User`.

6.3 Polymorphism

Polymorphism is used when the system works with a general `User` reference but performs different actions depending on the actual object type.

```

public void roleMenu() {
    if (currentUser instanceof Admin) {
        adminMenu();
    } else if (currentUser instanceof Student) {
        studentMenu();
    } else if (currentUser instanceof Teacher) {
        teacherMenu();
    } else if (currentUser instanceof Manager) {
        managerMenu();
    }
}

```

The same variable `currentUser` can represent different user types.

6.4 Abstraction

Abstraction is used through the `Researcher` interface. The system does not need to know the exact class of the researcher. It only needs to know that the object provides researcher behavior.

```
java
public interface Researcher extends Serializable {
    boolean addResearchPaper(ResearchPaper paper);
    List<ResearchPaper> getResearchPapers();
    String getResearcherName();
}
```

7. Enumerations

The project uses enums to represent fixed sets of values. This makes the code clearer and safer.

Enums are used for user roles, teacher titles, lesson types, request statuses, report types, room types, and research sorting options.

8. Custom Exceptions

The project uses custom exceptions to handle important business rules.

8.1 CreditLimitExceededException

This exception prevents students from registering for more than 21 credits.

```
public class CreditLimitExceededException extends Exception {
    public CreditLimitExceededException(String message) {
        super(message);
    }
}
```

```
if (credits + course.getCredits() > MAX_CREDITS) {
    throw new CreditLimitExceededException(
        "Student cannot register for more than " + MAX_CREDITS + " credits."
    );
}
```

8.2 LowHIndexException

```
if (year >= 4 && (supervisor == null || supervisor.calculateHIndex() < 3)) {
    throw new LowHIndexException(
        "Fourth-year student supervisor must have h-index at least 3."
    );
}
```

8.3 NonResearcherException

This exception is used when someone who is not a researcher tries to join a research project.

```
```java
if (!(candidate instanceof Researcher researcher)) {
 throw new NonResearcherException("Only researchers can join a research project.");
}
```

## 9. Design Patterns

### 9.1 Singleton Pattern

The `University` class uses the Singleton pattern. There should be only one university object in the system.

```
java
private static final University INSTANCE = new University();

private University() {
}

public static University getInstance() {
 return INSTANCE;
}
```

This ensures that all parts of the program work with the same shared university data.

### 9.2 Factory Pattern

`UserFactory` is used to create users based on `UserType`.

```
```java
public static User createUser(UserType type, int id, String username, String password, String email) {
    return switch (type) {
        case EMPLOYEE -> new Employee(id, username, password, email, 0.0, null);
        case ADMIN -> new Admin(id, username, password, email);
        case STUDENT -> new Student(id, username, password, email, 0.0, 1, 0, 0);
        case TEACHER -> new Teacher(id, username, password, email, 0.0, null, null);
        case MANAGER -> new Manager(id, username, password, email, 0.0, null, null);
    };
}
```

This separates object creation logic from the rest of the system.

9.3 Mediator Pattern

The system uses `MessageMediator` for communication between users.

```
```java
public interface MessageMediator {
 boolean sendMessage(User sender, User recipient, String content);
}
```

`University` implements this interface:

```

"""java
@Override
public boolean sendMessage(User sender, User recipient, String content) {
 if (sender == null || recipient == null || content == null || content.isBlank()) {
 return false;
 }

 Message message = new Message(sender, recipient, content);
 recipient.receiveMessage(message);
 addLog(sender, "DIRECT_MESSAGE_SENT to=" + recipient.getUsername());
 return true;
}
"""

```

This reduces direct dependency between users when sending messages.

#### 9.4 Data Store Pattern / Serialization

`DataStore` is responsible for saving and loading system state.

```

"""java
public static boolean saveState(University university) {
 try {
 Files.createDirectories(DATA_DIR);
 try (ObjectOutputStream output =
 new ObjectOutputStream(Files.newOutputStream(STATE_FILE))) {
 output.writeObject(university);
 }
 exportJsonSnapshot(university);
 return true;
 } catch (IOException exception) {
 university.addLog(null, "STATE_SAVE_FAILED " + exception.getMessage());
 return false;
 }
}
"""

```

This class separates data persistence logic from business logic.

#### 10. Authentication

The system requires users to log in with username and password. Authentication is handled by `AuthService`.

```

"""java
Optional<User> authenticatedUser = university.getUsers().stream()
 .filter(user -> user.isActive()
 && username.equalsIgnoreCase(user.getUsername())
 && user.checkPassword(password))
 .findFirst();
"""

```

If login is successful, the user is stored as `currentUser`. If not, a failed login is recorded in logs.

```

"""java
if (currentUser != null) {
 university.addLog(currentUser, "LOGIN");
} else {
 university.addLog(null, "FAILED_LOGIN " + username);
}
"""

```

## 11. Collections and Java API Usage

The project uses Java collections such as `ArrayList`, `HashSet`, `Deque`, `Optional`, and streams.

Examples:

```
java
private final List<Course> courses = new ArrayList<>();
private final Set<Researcher> participants = new HashSet<>();
private Deque<Message> inboxMessages = new ArrayDeque<>();
```

Streams are used for searching, filtering, and sorting.

```
java
return courses.stream()
 .filter(course -> code.equalsIgnoreCase(course.getCode()))
 .findFirst();
```

This makes the code shorter and easier to read.

## 12. UML Diagrams

The final report should include UML diagrams created in StarUML.

### Use Case Diagram

The use case diagram include these actors:

- Admin
- Student
- Teacher
- Manager
- Researcher

Main use cases:

- Login
- Manage users
- View courses
- Register for course
- View transcript
- Put marks
- Manage course
- Assign teacher
- Manage requests
- Add research paper
- Join research project
- View sorted research papers
- Save/load system data

### Class Diagram

The class diagram should show these relationships:

- `Student` extends `User`
- `Employee` extends `User`

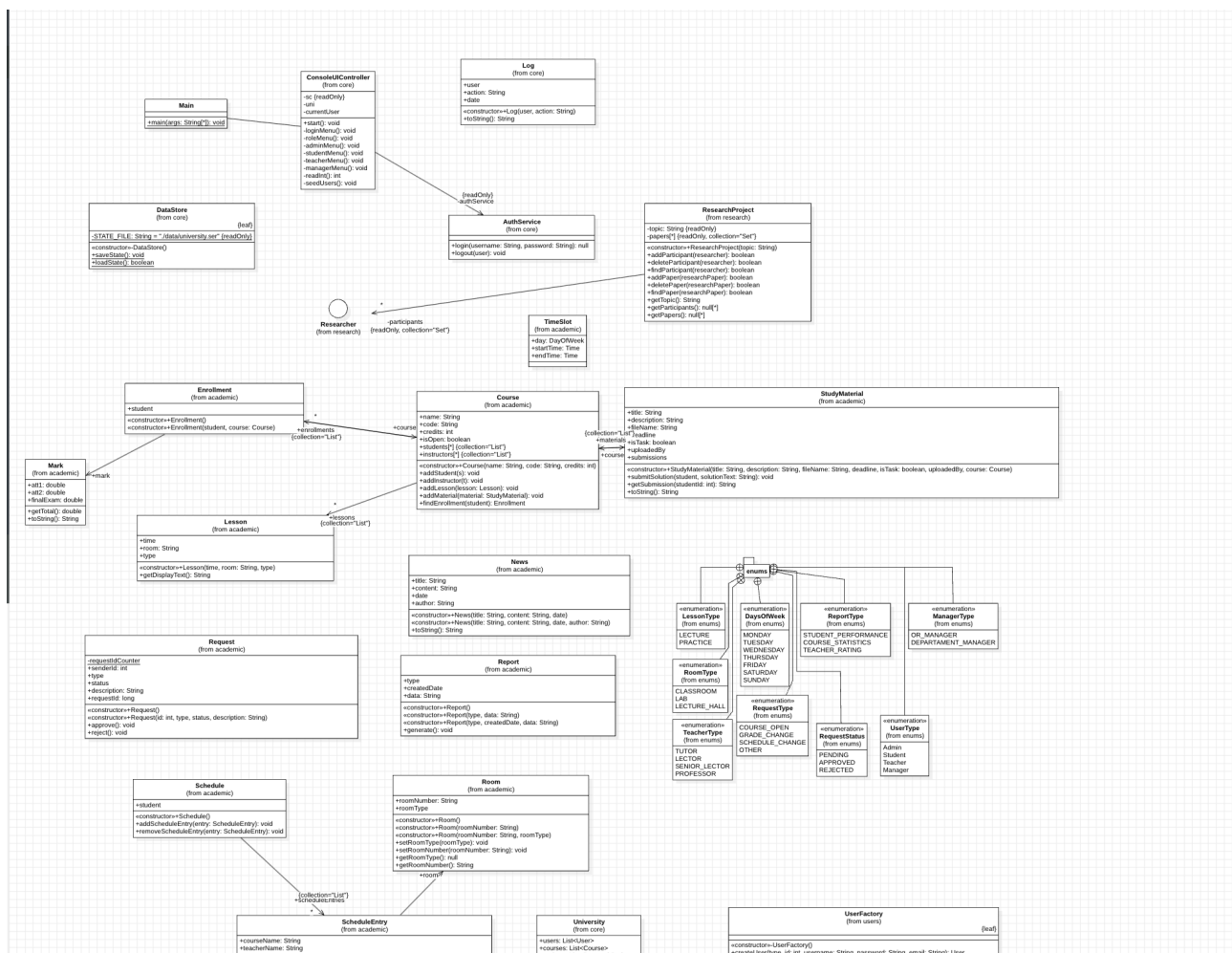
- 'Teacher' extends 'Employee'
- 'Manager' extends 'Employee'
- 'Admin' extends 'User'
- 'Teacher' implements 'Researcher'
- 'ResearchProject' has many 'Researcher' participants
- 'ResearchProject' has many 'ResearchPaper' objects
- 'Course' has many 'Teacher' instructors
- 'Course' has many 'Enrollment' objects
- 'Enrollment' connects 'Student', 'Course', and 'Marks'
- 'University' contains users, courses, researchers, projects, requests, and logs

Insert final UML screenshots here:

[Insert Use Case Diagram screenshot here]

[Insert UML Class Diagram screenshot here]

### 13. Project Management and Teamwork

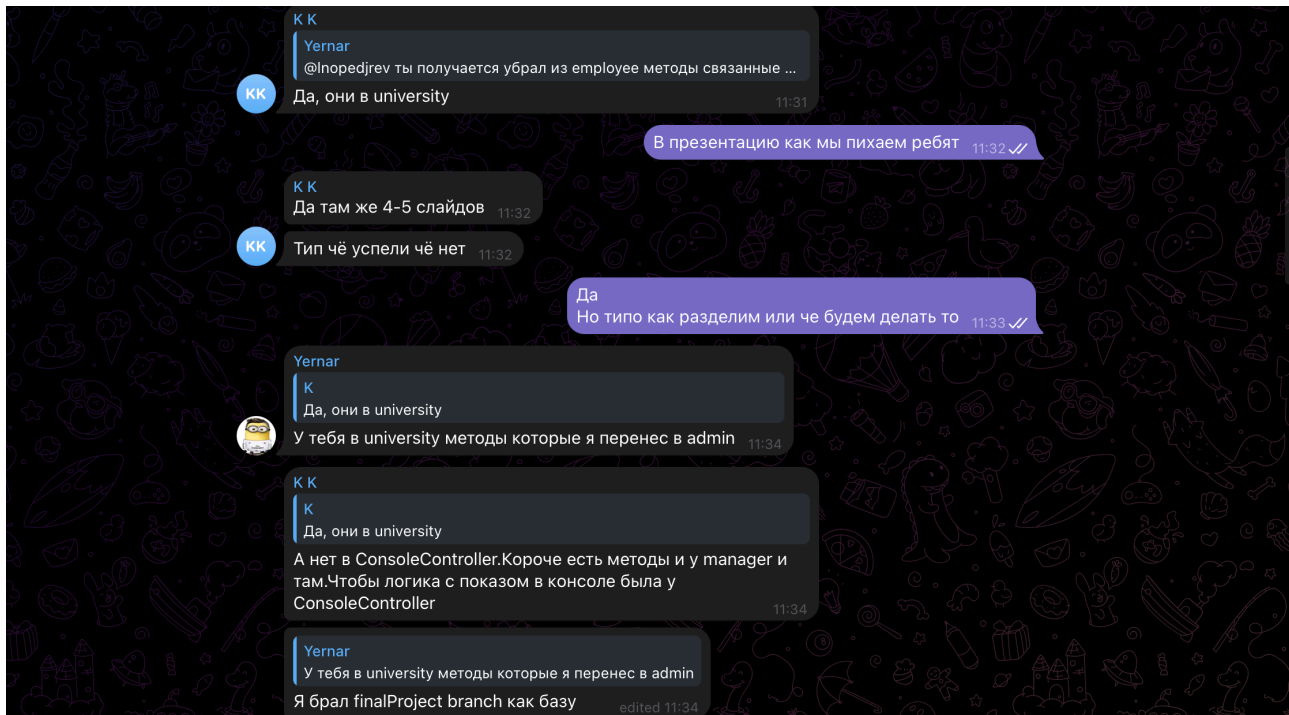


The project was divided between team members according to the main modules of the system.

Suggested team distribution:

- Semey Madiyar: user classes, inheritance hierarchy, authentication.
- Kirillov Grigoriy: academic module, courses, marks, transcript, lessons.
- Yessentur Nuraslanbek: research module, research papers, research projects, comparators.
- Team member 4: console UI, serialization, logs, testing, report and presentation.

The team communicated through Telegram or Teams during the project. The chat was used to discuss class responsibilities, UML diagrams, implementation problems, and final integration.



#### 14. Problems Faced During Development

During development, several problems appeared.

First, it was difficult to design the correct relationship between `Researcher`, `Teacher`, `Student`, and `Employee`. The requirement says that teachers and students can be researchers, but researcher is not always a separate user type. To solve this, we implemented `Researcher` as an interface.

Second, course registration required careful validation. A student should not register for the same course twice and should not exceed 21 credits. This was solved using checks and custom exceptions.

Third, research project participants had to be validated. Only researchers can join a project, so we created `NonResearcherException`.

Fourth, serialization required all saved objects to implement `Serializable`. Because the university stores many connected objects, the main model classes were made serializable.

Fifth, keeping UML diagrams consistent with code was challenging because the code changed during implementation. The diagrams had to be updated after the final version of the classes was completed.

#### 15. What Works

The following functionality works in the project:

- User authentication.
- Role-based console menus.
- Admin user management and logs.

- Student course registration.
- Credit limit validation.
- Transcript and marks viewing.
- Teacher course management and putting marks.
- Manager course registration management.
- Request management.
- Research papers and authors.
- Research project participants.
- Sorting research papers by citations, date, and length.
- Finding top cited researchers.
- Custom exceptions.
- Serialization and JSON snapshot export.
- Use of collections, enums, interfaces, inheritance, and polymorphism.

## 16. Limitations

The project also has some limitations:

- The system is console-based and does not have a graphical interface.
- Some advanced features, such as attendance and advanced schedule generation, are simplified.
- Some report generation features are basic.
- UML diagrams must be manually updated when the code changes.
- The system is designed mainly for demonstration and educational purposes, not for production use.

## 17. Conclusion

This project demonstrates the design and implementation of a research-oriented university information system using Java and Object-Oriented Programming. The system includes user roles, academic functionality, research functionality, authentication, exceptions, serialization, design patterns, and collections.

The most important requirements were implemented: course registration, marks, research papers, research projects, custom exceptions, enums, interfaces, and OOP principles. The project helped us understand how to design a larger Java application with multiple connected classes and responsibilities.

Overall, the system provides a structured model of a university and shows practical usage of OOP concepts in a real project.